# Collections

# Coleções

- Encadeamento
- Enumeration
- Coleções
- Iterator
- Tipos Genéricos
- Exemplos

Retirado do Tutorial Java

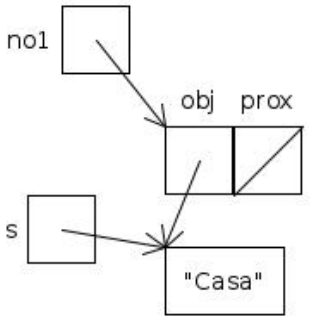
# Lista encadeada

```
class Lista {
    No inicio = null;

    void insereInicio(Object obj) {
        No n = new No(obj, inicio);
        inicio = n;
    }

    public String toString() {
        StringBuffer s = new StringBuffer();
        if (inicio != null) {
            s.append("(").append(inicio.obj).append(")");
            No p = inicio.prox;
            while (p != null) {
                s.append(" -> (").append(p.obj).append(")");
                p = p.prox;
            }
        }
        return s.toString();
    }
}
```

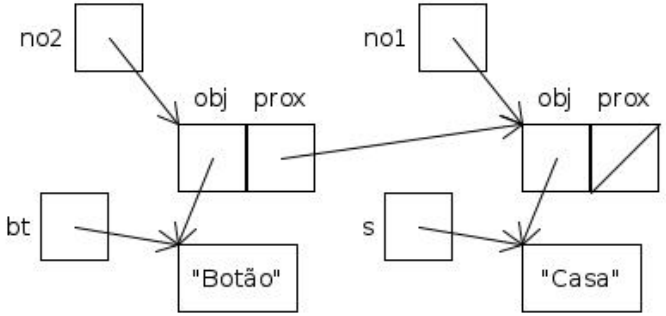
```
class No {
    Object obj;
    No prox;

    No(Object obj, No prox) {
        this.obj = obj;
        this.prox = prox;
    }
}
```

# Lista encadeada
# Classe No

```
class No {
    Object obj;
    No prox;
    No(Object obj, No prox) {
        this.obj = obj;
        this.prox = prox;
    }
}

...
String s = “Casa”;
No no1 = new No(s, null);
```

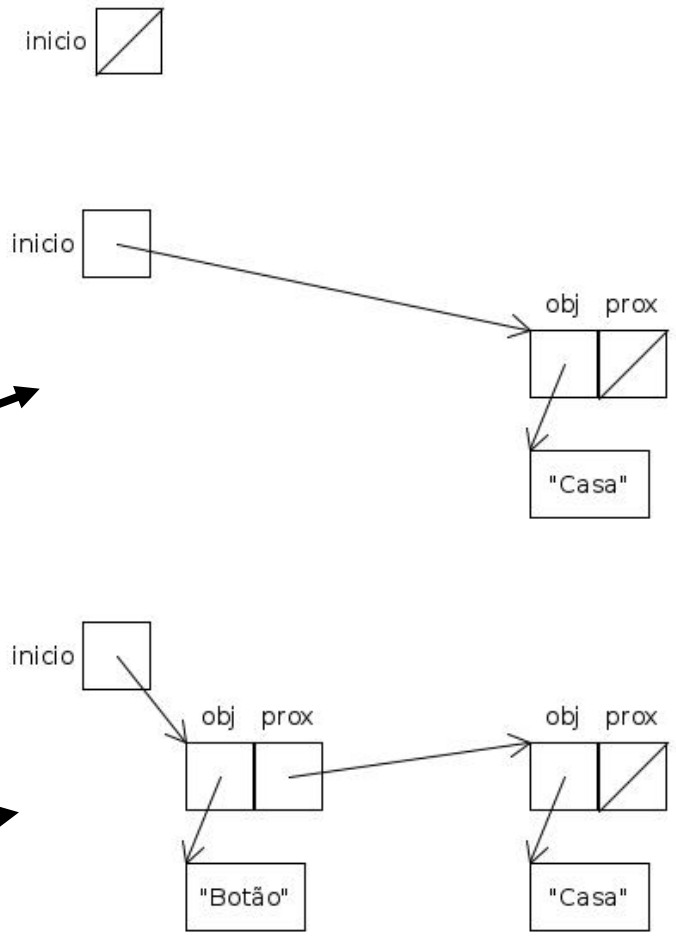



```
...
JButton bt = new JButton(“Botão”);
No no2 = new No(bt, no1);
```



# Lista encadeada

```
class Lista {
    No inicio = null;

    void insereInicio(Object obj) {
        No n = new No(obj, inicio);
        inicio = n;
    }
    ...
}

...
Lista l = new Lista();

...
String s = “Casa”;
l.insereInicio(s);

...
JButton bt = new JButton(“Botão”);
l.insereInicio(bt);
```



# Lista encadeada
## (continuação)

```
class No {...}

class Lista {...}

public class TestaLista {
   public static void main(String[] args) {
     Lista l = new Lista();

     for(int i=0; i<10; i++)
       l.insereInicio("Numero " + i);

     System.out.println(l);
   }
}
```

# Enumeration

- Um objeto que implementa Enumeration permite a "caminhada" nos elementos de uma coleção de dados sem conhecer a sua estrutura.
- Chamadas sucessivas ao método nextElement() retorna elementos sucessivos da coleção um de cada vez.

- Declaração da interface Enumeraton:

```
public interface Enumeration {
    boolean hasMoreElements();
    Object nextElement();
}
```

# Enumeration

## Exemplo usando a classe Lista anterior

```
class Lista implements Enumeration {
    void insereInicio(Object obj) { ... }

    No inicio = null, posAtual = null;

    public Enumeration elements() {
        posAtual = inicio;
        return this;
    }

    public boolean hasMoreElements() {
        return posAtual != null;
    }

    public Object nextElement() {
      Object obj = posAtual.obj;
      posAtual = posAtual.prox;
      return obj;
    }
}
```

# Exemplo de Enumeration (continuação)

```
import java.util.Enumeration;
...
class TestaListaEnumeration {
    public static void main(String[] args) {
      Lista l = new Lista();
      for(int i=0; i<10; i++)
        l.insereInicio("Numero " + i);

      System.out.println(l);

      Enumeration e = l.elements();
      while (e.hasMoreElements())
        System.out.println(e.nextElement());
    }
 }
```

# Coleções (Collections)

- Uma coleção é um conjunto de instâncias agrupados em uma estrutura de dados
- Até a versão 1.1 do Java
  - Vector, Hashtable e vetores
- Collections framework - introduzido na versão 1.2 (Java 2)
  - List, Set, Queue, Map
- Disponíveis no Java no pacote java.util
- Aprimorado na versão 1.5 com Generics

# Benefícios

- Reduz o esforço de programação
  - estruturas de dados e algoritmos variados prontos para usar
- Aumenta a qualidade e velocidade de execução do programa
  - implementações de alta qualidade e alto desempenho
- Permite interoperabilidade entre algumas APIs
  - várias APIs utilizam Collections para trocar dados

# Vector

- Uma lista encadeada pode ser substituída por uma instância de Vector
- Vector é mais lento que usar um vetor em Java, mas é bem versátil e prático
- Vector:
  - pode crescer dinamicamente
  - métodos podem inserir e remover elementos em qualquer posição

# Vector – exemplo

```
import java.util.Vector;

public class TestaVector{
   public static void main(String[] args) {
     Vector v=new Vector();

     for(int i=0; i<10; i++)
       v.add("Numero " + i);

     v.insertElementAt("inserido na posicao 5", 5);
     v.set(3, "posicao 3 alterada");

     for(int i=0; i<v.size(); i++)
       System.out.println(i + ":" + v.get(i));
   }
}
```

# Vector & Enumeration

- Exemplo:

```
import java.util.*;

public class TestaVectorEnumeration {
    public static void main(String[] args) {
        Vector v = new Vector();
        for (int i = 0; i < 10; i++)
            v.add("Numero " + i);

        Enumeration e = v.elements();
        while (e.hasMoreElements())
            System.out.println(e.nextElement());
    }
}
```

# Stack

- Stack (pilha) é uma classe descendente de Vector
- Exemplo:

```
import java.util.Stack;

public class TestaStack {
    public static void main(String[] args) {
        Stack s = new Stack();

        for (int i = 0; i < 10; i++)
            s.push("Numero " + i);

        while (!s.empty())
            System.out.println(s.pop());
    }
}
```

# Hashtable

- Uma Hashtable mapeia objetos chaves para valores
- Qualquer objeto não nulo pode ser usado como chave ou como valor
- Exemplo:

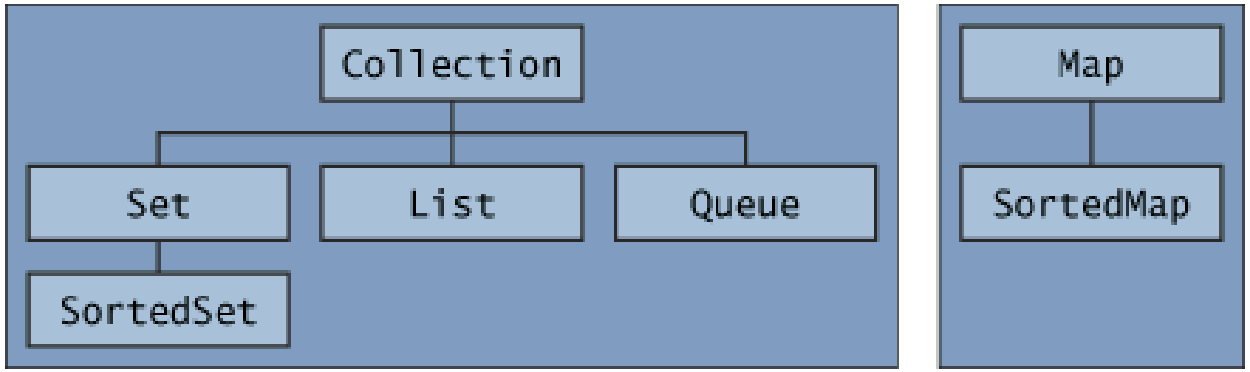
```
import java.util.Hashtable;

public class TestaHashtable {
    public static void main(String[] args) {
        Hashtable ht = new Hashtable();

        ht.put(new Float(1.23), "um e vinte tres");
        ht.put(2.34, "dois e trinta e quatro");
        ht.put("nome", "Rene");

        System.out.println(ht.get(new Double(2.34)));
        System.out.println(ht.get("nome"));
        System.out.println(ht.get(1.23F));
    }
}
```

# O que é o collections framework?

- Arquitetura unificada para representar e manipular coleções
  - Interfaces: permite que coleções sejam manipuladas independentemente dos detalhes da sua representação
  - Implementações: implementações concretas das interfaces de coleção
  - Algoritmos: métodos que realizam operações úteis, como buscas e ordenações

# Collection

- Permite o máximo de generalidade entre coleções
- Interface base para outras interfaces




- Garante a existência de Iterable que retorna um Iterator
  - next, hasNext, remove (opcional)

# Iterator

- Iterator é similar à Enumeration, mas difere em dois aspectos:
- O iterator
  - permite a remoção de elementos durante a iteração
  - os nomes dos métodos foram simplificados

```
public interface Iterator {
    boolean hasNext();
    Object next();
    void remove(); // Optional
}
```

# Iterator

## Exemplo usando a classe Lista anterior

```
import java.util.Iterator;

class Lista implements Iterator, Iterable {
    No inicio = null, posAtual = null, posAnterior = null;

    ...

    public Iterator iterator() {
        posAtual = null;
        posAnterior = null;
        return this;
    }

    public boolean hasNext() {
        if (inicio == null)
            return false;
        if (posAtual == null)
            return posAnterior == null;
        else
            return posAtual.prox != null;
    }
```

# Exemplo de Iterator (continuação – classe Lista)

```
    public Object next() {
        if (posAtual == null) {
            if (posAnterior == null)
                posAtual = inicio;
        } else {
            posAnterior = posAtual;
            posAtual = posAtual.prox;
        }
        return posAtual.obj;
    }

    public void remove() {
        if (inicio != null) {
            if (inicio == posAtual) {
                inicio = posAtual.prox;
                posAtual = null;
            } else
                posAnterior.prox = posAtual.prox;
        }
    }
}
```

# Exemplo de Iterator
## (continuação)

```
public class TestaListaIterator {
    public static void main(String[] args) {
        Lista l = new Lista();
        for (int i = 0; i < 2; i++)
            l.insereInicio("Numero " + i);

        for (Object n : l)
            System.out.println(": " + n);

        Iterator i = l.iterator();
        i.next();
        i.remove(); // remove primeiro nó

        i = l.iterator();
        while (i.hasNext())
            System.out.println(i.next());
    }
}
```

# Vector & Iterator

```
import java.util.*;

public class TestaVectorIterator {
    public static void main(String[] args) {
        Vector v = new Vector();
        for (int i = 0; i < 10; i++)
            v.add("Numero " + i);

        Iterator i = v.iterator();
        i.next();
        i.next();
        i.remove(); // remove o segundo elemento

        i = v.iterator();
        while (i.hasNext())
            System.out.println(i.next());
    }
}
```

# Iterator x Enumeration

- Iterator é o substituto do Enumeration no Java collections framework.
- Iterators são diferentes do Enumerations:
  - Iterator permite o chamador remover elementos de uma coleção durante a iteração com uma semântica bem definida
  - O nomes dos métodos foram melhorados

# Interfaces

- Permite que coleções sejam manipuladas independentemente dos detalhes da sua representação
  - Varias implementações usando estruturas de dados diferentes para cada interface
  - A escolha depende da aplicação
- Collection – raiz da hierarquia de coleções
- Set – uma coleção que não pode ter objetos duplicados
- List – é uma coleção onde a ordem é mantida, cada objeto pode ser manipulado através de seu índice
- Map – mapeia objetos chaves para objetos, não são permitidas chaves duplicadas (não é subinterface de Collection)

# List - implementações

- Vector
  - reformado para a versão 1.2 do Java implementando List
  - é synchronized (thread safe)
  - lista de objetos armazenados em um vetor interno
- ArrayList
  - lista de objetos armazenados em um vetor interno
- LinkedList
  - lista de objetos armazenados em uma lista encadeada
- ArrayList e LinkedList podem ser “thread save”
  - `List l = Collections.synchronizedList(new ...);`

# Vector x ArrayList

- Vector é ThreadSafe, ou seja, pode ser usado por várias linhas de execução concorrente, porém é mais lento que o ArrayList
- ArrayList tem melhor desempenho, mas não é ThreadSafe
- É possível tornar um ArrayList ThreadSafe, porém seu desempenho será prejudicado

```
List list = Collections.synchronizedList(new ArrayList(...));
```

# Set - implementações

- HashSet
  - conjunto de objetos armazenados em um vetor interno
- LinkedHashSet
  - armazenamento do conjunto de objetos em uma lista encadeada
- Não é permitido dois o1 e o2 objetos em um conjunto se o1.equals(o2).
- Implementações não synchronized, mas podem usar `Set s = Collections.synchronizedSet(new ...);`

# Map - implementações

- Hashtable
  - reformado para a versão 1.2 do Java implementando Map
  - é synchronized (thread safe)
  - lista de objetos armazenados em um vetor interno
- HashMap
  - lista de objetos armazenados em um vetor interno
  - Implementação não synchronized, mas pode ser

```
Map m = Collections.synchronizedMap(new HashMap( ... ));
```

# Subinterfaces com Ordenação

- O Iterator caminha pelos objetos ordenadamente
- O objetos inseridos devem
  - implementar a interface Comparable (ou especificar um Comparator)
  - ser comparados mutuamente usando
- o1.compareTo(o2) (ou Comparator.compare(o1, o2))
- SortedSet
  - descendente da interface Set
  - os elementos são mantidos ordenados na estrutura
- SortedMap
  - descendente da interface Map
  - ordem de ordenação definida pela chave

# SortedSet - implementação

- TreeSet
  - conjunto de objetos
  - armazenamento dos objetos ordenados
    - pela “ordem natural” dos objetos - classe dos objetos implementa Comparable
    - por uma ordem definida na instanciação do conjunto – utiliza uma classe que implementa Comparator
  - Operações básicas (add(), remove(), contains()) mais rápidas que os outros conjuntos O(log(n))

# SortedMap - implementação

- TreeMap
  - mapeia objetos chaves para objetos
  - armazenamento dos objetos ordenados
    - pela ordem dos objetos - classe dos objetos implementa Comparable
    - por uma ordem definida na instanciação do conjunto – utiliza uma classe que implementa Comparator

# Algoritmos

- A classe Collections disponibiliza algoritmos aplicados a classes que implementam Collection e List
- Diversos algoritmos:
  - sort() – ordena,
  - shuffle() – embaralha,
  - reverse() – ordena reversamente,
  - swap() – troca dois elementos em uma lista,
  - binarySearch() – busca binária
  - min() e max() – encontra limites extremos
  - ...

# Algoritmos – sort()

- Ordenação (ordem natural)

```
import java.util.*;

public class Sort {
  public static void main(String args[]){
    List l = Arrays.asList(args);
    Collections.sort(l);
    System.out.println(l);
  }
}
```

  - O Arrays.asList(args) transforma o vetor args em uma lista.
  - O Collections.sort() ordena os objetos da lista
  - Para determinar a ordem natural, as classes dos objetos na lista (String no caso) devem implementar Comparable

# Algoritmos – sort()

- Ordenação (ordem determinada pelo Comparator)

```
import java.util.*;

public class TestaSortComparator {
    public static void main(String args[]) {
        String s[] = { "Joao", "Maria", "Pedro", "Antonio" };
        List l = Arrays.asList(s);
        Collections.sort(l, new Comparador());
        System.out.println(l);
    }
}

class Comparador implements Comparator {
  public int compare(Object o1, Object o2) {
        return  ((String)o2).length() - ((String)o1).length();
  }
}
```

# Tipos Genéricos

- Introduzido na versão 1.5
- Semelhante a templates do C++
- Generics permite a abstração do tipo
- Exemplo de um problema solucionado por generics:

```
List l = new ArrayList();
l.add(new Float(1.23));
l.add(new Float(2.34));
Float f = (Float)l.iterator().next();
```

É necessário um cast (Float), para que a instância de Float armazenada como Object no interior da Lista, possa ser usada.

# Tipos Genéricos

- O exemplo anterior pode ser melhorado com generics usado na definição da List:

```
List<Float> l = new ArrayList<Float>();
l.add(new Float(1.23));
l.add(new Float(2.34));
Float f = l.iterator().next();
```

- A declaração de l na transparência anterior teve o tipo declarado em aberto
  - List<Float> indica que é uma lista de Float.

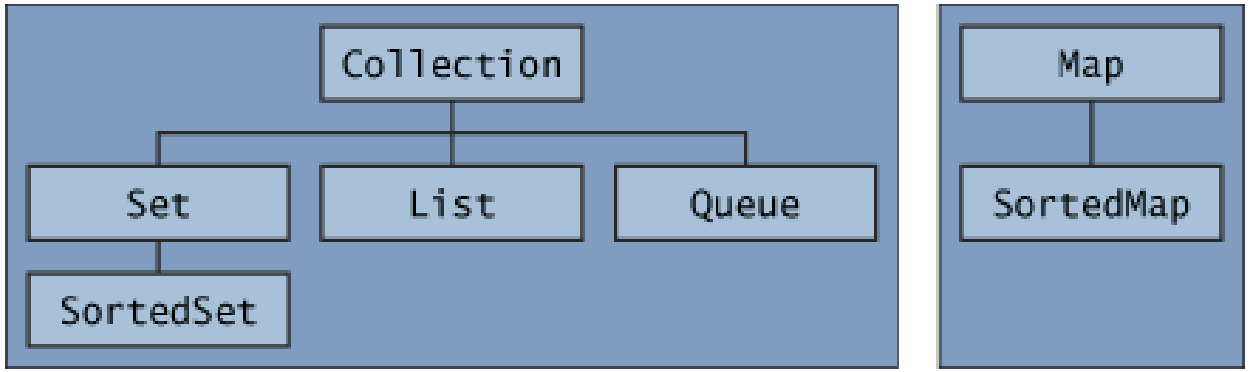
# Tipos Genericos
## Declaração

```
class No<ENo> {
    ENo obj;
    No<ENo> prox;

    No(ENo obj, No<ENo> prox) {
        this.obj = obj;
        this.prox = prox;
    }
};

class Lista<EList> {
    No inicio = null;

    void insereInicio(EList obj) {
      No n = new No<EList>(obj, inicio);
      inicio = n;
    }...
}
```

Tipo Genérico <E>,
E poderá assumir qualquer classe

# Collection com Tipos Genéricos

- Permite o máximo de generalidade com tipos específicos




- O tipo genérico especifica o tipo dos dados armazenados

# Queue - Fila

```
import java.util.*;

class Fila {
    public static void main(String s[]) {
        Queue<Integer> fila = new LinkedList<Integer>();
        fila.add(12);
        fila.add(new Integer(34));
        fila.add(new Integer(56));
        while (!fila.isEmpty()) {
            Integer i = fila.poll();
            System.out.println(i.intValue());
        }
    }
}
```

# Laço For Melhorado
# enhanced for-loop ou for-each

- O **for** pode ser simplificado nas Collections e nos vetores

```
int soma(int[] vet) {
    int soma = 0;
    for (int i: vet)
        soma += i;
    return soma;
}
```

# TreeSet - Conjunto

```
import java.util.*;
class Conjunto {
    Set<String> conj = new TreeSet<String>();

    Conjunto() {
        conj.add("Joao");
        conj.add("Maria");
        conj.add("Jose");
        conj.add("Pedro");
        conj.add("Maria");
        for (String s: conj) System.out.println(s);
    }

    public static void main(String s[]) {
        new Conjunto();
    }
}
```

# TreeSet - Conjunto

```
import java.util.*;

class Conjunto1 {
    class Compara implements Comparator<String> {
        public int compare(String s1, String s2) {
            return s1.length() - s2.length();
        }
    }

    Conjunto1() {
        Set<String> conj = new TreeSet<String>(new Compara());
        conj.add("Maria");
        conj.add("Joao");
        conj.add("Jose");
        conj.add("Pedro");
        conj.add("Antonio");
        for (String s : conj)
            System.out.println(s);
    }

    public static void main(String s[]) {
        new Conjunto1();
    }
}
```

# TreeMap - Dicionário

```
import java.util.*;

class Dicionario {
    public static void main(String s[]) {
        Map<String, Float> conj = new TreeMap<String, Float>();
        conj.put("Joao", new Float(7.5));
        conj.put("Maria", new Float(9.5));
        conj.put("Jose", new Float(6.0));
        conj.put("Pedro", new Float(8.3));
        conj.put("Maria", new Float(10.0));
        System.out.println("Jose vale: " + conj.get("Jose"));
        System.out.println("JOSE não existe: " + conj.get("JOSE"));
        for (Map.Entry<String, Float> p : conj.entrySet()) {
            System.out.println(p.getKey() + " " +
                    p.getValue());
        }
    }
}
```